ROS 2 与 Nav2 麦克纳姆轮小车运动控制教程

部署与使用手册 · OMOVE 仿真工程

Ubuntu 22.04 / ROS 2 Humble / Gazebo Classic

版本 V1.0　　2026 年 10 月 30 日

本教程面向已有 omove_description 源码的使用者，按“部署—底盘验证—建图—地图导航—验收”的顺序跑通系统。默认工作空间为 /home/deng/omove_ws；迁移到其他电脑时，应统一替换示例中的工作空间路径。本文覆盖仿真部署和使用，不包含操作系统重装、实体底盘驱动与实车标定。

目　录

一、系统组成与运行约定

二、部署前检查

三、依赖安装与编译

四、配置文件与接口核对

五、首次底盘与数据链路验证

六、建图操作步骤

七、导航操作步骤

八、已有底盘的独立导航启动

九、完整验收与自动测试

十、常见故障与排查顺序

十一、文件归档与日常运行

一、系统组成与运行约定

1. 完整流程

```
准备 ROS 2 环境 → 安装依赖 → 放置源码 → 编译并加载工作空间
                ↓
检查四轮控制、激光、里程计与 TF
                ↓
SLAM 建图 → 键盘扫描 → 保存 YAML/PGM
                ↓
结束建图 → 加载地图启动 Nav2 → 初始化 AMCL
                ↓
发送目标 → 路径规划 → 局部控制 → 四轮运动 → 到达
```

建图使用 SLAM Toolbox，已有地图定位使用 AMCL；两种模式分别启动。地图保存后，导航模式不需要继续运行 SLAM。

2. 运行入口

入口	自动启动的内容	使用场景
gazebo.launch.py	Gazebo、模型、四轮控制、关节反馈、里程计	单独测试底盘
slam_sim.launch.py	房间仿真、底盘、SLAM Toolbox	重新扫描地图
nav_sim.launch.py	房间仿真、底盘、地图服务、AMCL、Nav2	完整导航入口
nav_bringup.launch.py	地图服务、AMCL、Nav2	底盘和传感器已单独启动

正常操作只选一个完整入口。不要同时运行 slam_sim 与 nav_sim，不要在完整入口之外再重复启动 gazebo.launch.py。

3. 终端与环境约定

建议使用三个终端。每个新终端都先加载环境：

```
source /opt/ros/humble/setup.bash
source /home/deng/omove_ws/install/setup.bash
```

首次尚未编译时，先只加载 ROS 环境；install/setup.bash 在成功构建后产生。

模式	终端 1	终端 2	终端 3
底盘测试	Gazebo 与底盘	话题、反馈、TF 检查	速度控制
建图	SLAM 仿真入口	RViz 查看地图和激光	键盘扫描、保存地图
导航	Nav2 仿真入口	RViz 初始化并发送目标	检查控制器、节点和结果

保持正在运行的启动终端，不要在其中继续输入其他命令。Ctrl+C 结束当前前台进程；关闭导航前先取消未完成目标，关闭建图前先确认地图已保存。

二、部署前检查

1. 主机与软件

使用 Ubuntu 22.04、ROS 2 Humble Desktop 和能够运行 Gazebo 的图形环境。检查：

```
lsb_release -a
source /opt/ros/humble/setup.bash
echo "$ROS_DISTRO"
command -v ros2
```

ROS_DISTRO 应为 humble。不要在同一终端混合加载不同发行版。本文以已经装好 Humble 和相应软件源为前提；若 /opt/ros/humble 不存在，先完成 ROS 2 Humble 系统安装，再执行后续步骤。

2. 源码与数据

将项目完整源码放在工作空间 src 下。教程不假定存在公开仓库地址，不要用其他项目同名包替代。

```
/home/deng/omove_ws/
├── src/omove_description/
│   ├── package.xml
│   ├── CMakeLists.txt
│   ├── launch/
│   ├── config/
│   ├── urdf/
│   ├── worlds/
│   ├── rviz/
│   ├── maps/
│   ├── omove_description/
│   ├── scripts/
│   └── test/
└── maps/                        # 自己扫描得到的地图
```

源码包内 maps/room.yaml 为默认地图；工作空间 maps/room_new.yaml 为本教程新建地图。两个目录用途不同，导航时明确传入需要的地图。

3. 通信环境一致性

所有终端应使用相同 ROS_DOMAIN_ID、RMW_IMPLEMENTATION 与 ROS_LOCALHOST_ONLY，RViz 也包括在内。先检查现有值，不必为单机运行额外调内核参数：

```
printenv ROS_DOMAIN_ID
printenv RMW_IMPLEMENTATION
printenv ROS_LOCALHOST_ONLY
```

若需要指定 domain，在所有参与终端统一设置，例如 export ROS_DOMAIN_ID=0，再重新启动进程。不要只在一个终端改。默认 RMW 已正常时无需更换；如果项目已使用 Cyclone DDS，应确认该包已安装并保持所有终端一致。

三、依赖安装与编译

1. 安装构建工具和功能组件

已有完整环境可跳过重复安装。下面命令安装本教程直接使用的组件：

```
sudo apt update
sudo apt install -y \
  build-essential cmake python3-yaml \
  python3-colcon-common-extensions python3-rosdep \
  ros-humble-xacro ros-humble-robot-state-publisher \
  ros-humble-gazebo-ros-pkgs ros-humble-gazebo-ros2-control \
  ros-humble-ros2-control ros-humble-ros2-controllers \
  ros-humble-navigation2 ros-humble-nav2-bringup \
  ros-humble-nav2-map-server ros-humble-slam-toolbox \
  ros-humble-rviz2 ros-humble-teleop-twist-keyboard
```

若选择 Cyclone DDS，再安装 ros-humble-rmw-cyclonedds-cpp。安装命令失败时先处理软件源或网络问题，不应忽略报错后继续编译。

2. 初始化依赖规则

首次部署且尚未初始化 rosdep 时执行：

```
sudo rosdep init
rosdep update
```

如果提示规则已存在，不必重复初始化，只执行 rosdep update。安装工程声明的依赖：

```
cd /home/deng/omove_ws
source /opt/ros/humble/setup.bash
rosdep install --from-paths src --ignore-src -r -y
```

3. 构建与加载

```
cd /home/deng/omove_ws
colcon build --packages-select omove_description
source install/setup.bash
```

成功标志为 Finished <<< omove_description 和 Summary。出现 Failed 时先解决错误。当前项目主要安装 Python 程序、模型和配置，构建成功并不能证明 TF 或物理运动正确，还要完成后续运行检查。

确认安装文件可见：

```
ros2 pkg prefix omove_description
ros2 pkg executables omove_description
ls install/omove_description/share/omove_description/rviz/
ros2 launch omove_description nav_sim.launch.py --show-args
```

应能找到 mecanum_kinematics.py、mecanum_odometry.py、validate_navigation.py，以及 omove_navigation.rviz。

4. 修改后的更新方式

编辑 src 中的模型、参数、启动文件或 RViz 配置后，先结束相关进程，再重新构建、加载环境并启动。当前采用普通安装，不应假定修改源码后 install 自动更新。

```
cd /home/deng/omove_ws
source /opt/ros/humble/setup.bash
colcon build --packages-select omove_description
source install/setup.bash
```

四、配置文件与接口核对

1. 配置位置

文件	核对内容
urdf/omove.urdf.xacro	底盘尺寸、轮轴方向、滚子手性、惯量和关节接口
urdf/gazebo.urdf.xacro	激光插件、/scan 映射、控制器 YAML 路径
config/mecanum_controllers.yaml	四轮关节列表、控制器类型、状态广播器
omove_description/mecanum_kinematics.py	逆运动学、尺寸参数、超时和轮速限幅
omove_description/mecanum_odometry.py	反馈匹配、正运动学、odom 与 base frame
config/slam_params.yaml	scan、map/odom/base 坐标、激光范围和建图分辨率
config/nav2_params.yaml	AMCL、NavFn、DWB、速度平滑、costmap 与 footprint
rviz/omove_navigation.rviz	地图、模型、激光 QoS 和 Navigation 2 面板
worlds/room.world	真实仿真障碍与地图场景是否一致

2. Topic 契约

Topic/动作	类型	用途
/cmd_vel	geometry_msgs/msg/Twist	运动学节点接收的最终车体速度
/cmd_vel_nav	geometry_msgs/msg/Twist	Nav2 控制到速度平滑之间的速度接口
/front_left_velocity_controller/commands	std_msgs/msg/Float64MultiArray	前左轮角速度，单元素数组
/front_right_velocity_controller/commands	std_msgs/msg/Float64MultiArray	前右轮角速度，单元素数组

/rear_left_velocity_controller/commands	std_msgs/msg/Float64MultiArray	后左轮角速度，单元素数组
/rear_right_velocity_controller/commands	std_msgs/msg/Float64MultiArray	后右轮角速度，单元素数组
/joint_states	sensor_msgs/msg/JointState	主动轮和被动滚子关节状态
/odom	nav_msgs/msg/Odometry	轮速里程计，child_frame_id=base_footprint
/scan	sensor_msgs/msg/LaserScan	二维激光扫描，frame=laser_link
/map	nav_msgs/msg/OccupancyGrid	建图或已加载的静态地图
/initialpose	geometry_msgs/msg/PoseWithCovarianceStamped	初始定位
/plan	nav_msgs/msg/Path	全局路径
/navigate_to_pose	nav2_msgs/action/NavigateToPose	目标、反馈与结果

二维 LaserScan 可在 RViz 中显示为扫描点，但不需要三维 PointCloud2、PCD 地图或点云转栅格流程。本系统直接保存二维 YAML/PGM。

3. TF 契约

```
map → odom → base_footprint → base_link → laser_link
                                  ↳ 四轮 → 被动滚子
```

变换	发布者
map → odom	建图为 SLAM Toolbox；导航为 AMCL，二选一
odom → base_footprint	mecanum_odometry
base_footprint → base_link → laser_link	robot_state_publisher，来自 URDF 固定关节
base_link → wheel → roller	robot_state_publisher，结合 joint_states

仿真使用 /clock，相关节点和 RViz 的 use_sim_time 应为 true。重新启动 Gazebo 后，建议重新打开 RViz，清除上一轮时间缓存。

4. 全向控制参数

底盘 X 向前、Y 向左、Z 向上。只使用 Twist 的 linear.x、linear.y、angular.z。设 r=0.076 m、a=0.18+0.20=0.38 m，轮速顺序为前左、前右、后左、后右：

```
w_FL = (vx - vy - a*wz) / r
w_FR = (vx + vy + a*wz) / r
w_RL = (vx + vy - a*wz) / r
w_RR = (vx - vy + a*wz) / r
```

每轮 12 个可自由转动的 45° 滚子，轮关节与滚子手性按以上符号约定配置。轮速里程计根据反馈计算 vx、vy、wz，并按消息时间积分；它不是 Gazebo 真实位置，空转和接触误差会造成偏差。

Nav2 配置要同时允许 DWB 与速度平滑器的 Y 方向运动；只改其中一处可能无法横移。

参数	当前值
AMCL 运动模型	nav2_amcl::OmniMotionModel
前后速度范围	−0.15～0.20 m/s
横向速度范围	−0.15～0.15 m/s
最大角速度	0.6 rad/s
footprint	0.54×0.46 m 矩形，加 0.01 m padding
GridBased 规划容差	0.1 m
位置/朝向到点容差	0.25 m / 0.25 rad
ros2_control 控制器管理器更新频率	100 Hz
Nav2 局部控制频率	20 Hz
轮速上限	12 rad/s，四轮等比例缩放
指令超时	0.5 s，检查周期 0.05 s

超时检查使用单调时钟；进程正常运行但收不到指令时输出零速。它不覆盖进程崩溃或控制器故障。修改几何尺寸时，运动学、里程计、URDF 与 footprint 必须一起核对。

五、首次底盘与数据链路验证

1. 启动基础仿真

每个终端先加载环境。终端 1:

```
ros2 launch omove_description gazebo.launch.py
```

默认空世界，不含 SLAM 或 AMCL。若直接准备建图，可跳过该入口，按第六节启动房间建图；不要重复启动。无界面测试可加 gui:=false。

2. 检查控制器与反馈

终端 2:

```
ros2 control list_controllers
ros2 topic echo /joint_states --once
ros2 topic echo /odom --once
ros2 topic echo /scan --once --field header
ros2 run tf2_ros tf2_echo odom base_footprint
```

五个控制器都应为 active：joint_state_broadcaster 与四个 velocity_controller。TF 命令会持续输出，Ctrl+C 结束检查。此模式没有 map 坐标，不能以 map 缺失判断底盘失败。

3. 低速运动检查

终端 3 运行下列命令，观察 Gazebo 真实小车前进，再 Ctrl+C 停止，等待超时停车：

```
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.1}, angular: {z: 0.0}}'
```

分别测试横移与转向，每条结束后再执行下一条：

```
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {y: 0.1}, angular: {z: 0.0}}'
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 0.0}, angular: {z: 0.3}}'
```

检查方向是否正确、真实位置是否变化、停止发布后轮速是否归零。不能仅凭 /odom 或 RViz 模型变化判定横移成功。测试后 Ctrl+C 结束仿真再进入建图。

六、建图操作步骤

1. 建图前准备

关闭旧 Gazebo、导航、遥控与 RViz。确认 maps 输出目录可写，为新地图选择独立名称。改变 world 障碍布局后，应重新扫描，不要继续使用几何不一致的旧地图。

2. 终端 1：启动房间、底盘和 SLAM

```
source /opt/ros/humble/setup.bash
source /home/deng/omove_ws/install/setup.bash
ros2 launch omove_description slam_sim.launch.py
```

等待模型和控制器初始化，保持该终端运行。本入口加载房间，在世界坐标 (1.5,0) 附近生成机器人，并启动 online_async SLAM。

3. 终端 2：确认建图后打开 RViz

```
source /opt/ros/humble/setup.bash
source /home/deng/omove_ws/install/setup.bash
ros2 node list
ros2 run tf2_ros tf2_echo map laser_link
```

应有 /slam_toolbox，并持续输出 map 到 laser_link 的变换。Ctrl+C 结束 TF 检查，再执行：

```
rviz2 \
  -d /home/deng/omove_ws/install/omove_description/share/omove_description/rviz/
omove_navigation.rviz \
  --ros-args -p use_sim_time:=true
```

应看到地图、机器人和激光。若手工配置，按下表设置：

显示项	配置
Global Options / Fixed Frame	map
RobotModel	Description Source=Topic，/robot_description，Transient Local

Map	/map，Reliable，Transient Local
LaserScan	/scan，Best Effort，Volatile

Transient Local 用于接收此前已发布的地图或模型描述。激光处理范围约 0.16～8 m，每帧 360 线，地图分辨率 0.05 m。建图模式不要点 2D Pose Estimate，也不发 Nav2 Goal。

4. 终端 3：键盘扫描

```
source /opt/ros/humble/setup.bash
source /home/deng/omove_ws/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -p speed:=0.1 -p turn:=0.3
```

点击终端 3 使其获得键盘焦点。

按键	动作
i	前进
,	后退
j / l	左转 / 右转
k	停止

持续移动需要按住按键，通过系统键盘重复发送命令；首次按键重复延迟可能使小车短暂停顿。低速绕中央箱体扫描四周墙面，转角慢转，尽量回到起点附近。观察地图是否存在明显重影、断墙或撕裂。

5. 停车并保存地图

按 k 停车，再 Ctrl+C 退出键盘控制。SLAM 终端继续运行。终端 3 执行：

```
mkdir -p /home/deng/omove_ws/maps
ros2 run nav2_map_server map_saver_cli \
  -f /home/deng/omove_ws/maps/room_new \
  --ros-args -p save_map_timeout:=10.0
ls -lh /home/deng/omove_ws/maps/room_new.*
```

成功应提示 Map saved successfully，并生成 room_new.yaml 和 room_new.pgm。不要在保存完成前关闭 SLAM。同名保存会覆盖旧成果，需要保留时使用新名称。

6. 核对地图资料

YAML 中 image 指向配套 PGM，resolution 给出米/像素，origin 给出图像在地图坐标中的位置。两文件应一同归档，保持 image 路径有效。origin 不是机器人位置；新建地图原点也不一定等于 Gazebo 世界原点。

地图存储目录可直接通过绝对路径传给 Nav2，无需复制进源码包，也无需为新增地图重新编译。

七、导航操作步骤

1. 结束建图并检查地图

在终端 1 Ctrl+C 结束 SLAM，等待 Gazebo 退出，关闭旧 RViz；结束所有遥控。导航与遥控不能同时争用 /cmd_vel。

```
ls -lh /home/deng/omove_ws/maps/room_new.yaml \
       /home/deng/omove_ws/maps/room_new.pgm
```

2. 终端 1：加载新地图启动导航

```
cd /home/deng/omove_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch omove_description nav_sim.launch.py \
  map:=/home/deng/omove_ws/maps/room_new.yaml
```

此入口包括 Gazebo、底盘、map_server、AMCL 和 Nav2，不需要再单独启动 nav_bringup。等待约 15～20 s；滚子接触仿真速度取决于主机性能，应以节点和数据就绪为准。

如需显式指定参数文件：

```
ros2 launch omove_description nav_sim.launch.py \
  map:=/home/deng/omove_ws/maps/room_new.yaml \
  params_file:=/home/deng/omove_ws/src/omove_description/config/nav2_params.yaml
```

自定义场景可传 world:=绝对路径，但必须与地图一致。未传 map 时使用随包默认 room.yaml，而不是 room_new。

3. 终端 2：启动 RViz 并初始化

加载环境后执行第六节的 RViz 命令，确认 Navigation 2 面板存在。缺失时选择 Panels→Add New Panel→nav2_rviz_plugins→Navigation 2。

点击 2D Pose Estimate，在新地图中机器人实际所在位置按住左键，沿实际车头方向拖出箭头再松开。等待激光与墙面及箱体边缘基本重合。

Gazebo 的默认出生位置为世界坐标 (1.5,0)，但新地图坐标可能完全不同。应通过环境相对位置判断，而不是把世界坐标原样当作地图初始坐标。初次定位前 map→odom 缺失是可能的，初始化后必须建立。

4. 终端 3：确认导航就绪

```
source /opt/ros/humble/setup.bash
source /home/deng/omove_ws/install/setup.bash
ros2 control list_controllers
ros2 lifecycle get /amcl
ros2 lifecycle get /bt_navigator
ros2 run tf2_ros tf2_echo map base_footprint
```

五个底盘控制器应为 active；AMCL、BT Navigator 应为 active [3]；TF 应持续输出。停止检查命令后再发送目标。

5. 在 RViz 发送目标

点击 Nav2 Goal，在空闲目标点按住鼠标左键，拖出到达朝向箭头后松开。目标完成前不重复点新目标；初次先选择约 0.5 m 的短距离目标。

工具	正确用途	真实运动
2D Pose Estimate	设置机器人“现在在哪”	不移动，只改变定位估计
Nav2 Goal	设置机器人“要去哪里”	Nav2 执行时驱动底盘

Navigation 2 面板负责配合 Nav2 Goal 发送动作目标，不能只保留按钮而删去面板。观察面板是否进入执行、成功或失败状态。若只见 RViz 小车跳位、Gazebo 不动，先确认没有误用初始定位工具。

需要显示路径时，Add→Path，Topic 选择 /plan。保存配置应在 RViz 窗口内按 Ctrl+S 或 File→Save Config。使用安装目录配置时，若需长期保留编辑，应另存到源码 rviz/omove_navigation.rviz，再构建，避免下一次安装覆盖修改。

6. 终端发送目标（可选）

已通过 RViz 完成定位后，可以用动作命令替代鼠标操作。下面 (0.5,0) 只是地图坐标示例，应先在 RViz 确认该点可达，再替换为实际目标：

```
ros2 action send_goal /navigate_to_pose \
  nav2_msgs/action/NavigateToPose \
  '{pose: {header: {frame_id: map}, pose: {position: {x: 0.5, y: 0.0}, orientation: {w:
1.0}}}}' \
  --feedback
```

成功状态为 SUCCEEDED。0.25 m 到点容差允许停在目标附近，不代表精确到点。不要把 Ctrl+C 结束动作命令输出当作可靠的取消导航操作；需要取消时使用 Navigation 2 面板的取消功能并确认停车。

7. 正常停止

先在 Navigation 2 面板取消正在执行的目标，确认小车停稳；再 Ctrl+C 停止终端 1 的完整仿真导航入口，最后关闭 RViz。停止遥控时先按 k。不要在 SLAM 地图还未保存时直接关机。

八、已有底盘的独立导航启动

底盘、激光和里程计已经单独启动时，可以只启导航：

```
ros2 launch omove_description nav_bringup.launch.py \
  map:=/home/deng/omove_ws/maps/room_new.yaml \
  use_sim_time:=true
```

该入口不生成机器人、不启动电机控制器，也不产生 /scan 或 /odom。运行前必须保证接口与第四节一致，已有定位节点若同时发布 map→odom，则不能再启动竞争的 AMCL。

只有接入真实硬件并完成控制接口、反馈、TF 和时钟适配后，才能考虑 use_sim_time:=false。单独关闭仿真时间不等于实车部署完成。

九、完整验收与自动测试

1. 从关闭状态复现

完全退出旧进程，用保存的文件重新启动；依次完成新地图加载、RViz 初始定位、控制器 active 检查和三个导航目标。首次手动定位属于正常流程，连续测试之间不反复移动初始定位。

测试	操作	验收依据
前进	前方约 0.5 m 空闲点	动作成功，Gazebo 真实车体到达
侧向	侧方约 0.5 m，目标朝向接近当前朝向	真实位移发生，不只看轮速里程计
绕障	箱体另一侧，直线穿过箱体	路径绕行，未穿过箱体，最终成功

目标与墙面、箱体留充足距离。侧向目标允许 Nav2 选择组合平移与转向；要求纯横移时应用第五节速度实验单独检查。

2. 单元测试

```
cd /home/deng/omove_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
colcon test --packages-select omove_description
colcon test-result --verbose
```

也可直接运行：

```
python3 -m unittest discover -s src/omove_description/test -v
```

当前测试覆盖超时停车、异常输入、轮速限幅、滚子模型、里程计积分、时间重置和轮廓几何检查。通过单元测试后仍须进行实际运动验收。

3. 集成回归

关闭所有其他 Gazebo 实例后：

```
ros2 run omove_description validate_navigation.py \
  --output /tmp/omove_navigation_validation.json --domain 97
```

脚本自行启动默认房间、使用默认 room 地图、初始化定位并执行绕障、连续目标、占据点拒绝、Spin 行为和后续导航。报告包含动作状态、真实/定位轨迹、误差及采样轮廓间距；进程日志位于报告旁边。

该脚本默认坐标对应随包地图，不直接适用于原点不同的 room_new。每个目标墙钟超时默认 150 s，可用 --goal-timeout 调整。几何采样检查不能代替连续接触检测；单独 Spin 成功不等于全部自动恢复分支都已验证。

十、常见故障与排查顺序

1. 优先检查命令

每次先加载 ROS 与工作空间环境，然后按“节点—数据—TF—生命周期—速度—物理模型”顺序检查：

```
ros2 node list
ros2 topic list
ros2 topic info /scan --verbose
ros2 topic echo /scan --once --field header
ros2 topic echo /odom --once
ros2 run tf2_ros tf2_echo map laser_link
ros2 control list_controllers
ros2 lifecycle get /bt_navigator
ros2 topic echo /cmd_vel
```

持续命令 Ctrl+C 退出。无目标时 /cmd_vel 不持续输出并不能直接判定故障，需要结合目标状态。

2. 显示和定位

现象	优先检查	处理
map 坐标不存在	SLAM/AMCL、初始化、重复 TF	建图查 slam_toolbox；导航完成初始定位
Map 宽高为 0	发布者、Topic、Durability	/map + Transient Local
RobotModel 不显示	描述源、描述话题、关节 TF	Topic /robot_description，保留历史消息
激光不显示、队列满	QoS、TF、仿真时间	Best Effort；map→laser_link；use_sim_time=true
激光整体偏移	初始位置	保持静止后修正初始定位
沿墙越远越偏	初始朝向	修正朝向，观察多面墙同时匹配
只有部分墙能对齐	地图变形或场景不一致	查地图版本、扫描质量、world 布局
重启后异常	/clock 回退和旧缓存	随新仿真重新打开 RViz

3. 运动和目标

现象	优先检查	处理
Nav2 Goal 无响应	Navigation 2 面板、拖动释放、active	补面板并确认目标状态

RViz 跳位但真实车体不动	是否点击了 2D Pose Estimate	恢复真实定位，再用 Nav2 Goal
路径存在但不动	/cmd_vel_nav、/cmd_vel、控制器	查平滑器、订阅关系、active、限速
轮子转但无横移	实际加载模型和滚子接触	确认是滚子版模型，比较真实位姿
速度断续	指令发送频率和超时	连续控制用固定频率或键盘重复
两个显示位置不一致	定位估计、里程计空转、视角	对照真实位姿和激光，不按截图方向猜坐标
近障碍目标失败	footprint、膨胀和目标占据	选更远空闲点，不盲目缩小机器人轮廓

4. 部署和地图

现象	处理
找不到包或可执行程序	检查 src 位置、构建结果、每终端 source
修改不生效	对照 install 文件，重新构建并重启相关进程
保存地图超时	保持 SLAM 运行，确认 /map 后再保存
地图加载失败	查 YAML 的 image、绝对路径和文件权限
终端之间发现不同节点	对齐 ROS domain、RMW 和网络配置
Gazebo 冲突或重复实体	结束旧完整启动进程，只运行一套仿真

十一、文件归档与日常运行

每轮建图归档 YAML、PGM、场景版本、参数和采集日期。地图文件必须配对；不同布局和地图版本不要混用。不要依赖 /tmp 保存长期成果。

建议归档结构：

```
maps/
└── room_YYYYMMDD/
    ├── map.yaml
    ├── map.pgm
    ├── nav2_params.yaml
    └── 说明.txt              # 场景、初始位置说明、测试结果
```

日常已有地图导航只需要：终端 1 启动 nav_sim 并显式传 map，终端 2 打开 RViz、初始化并发目标，终端 3 按需检查。无需重新编译未修改的源码，也无需重新建图。

教程适用范围为本项目的静态场景仿真。实车、动态障碍、长时间耐久和高精度到点需要独立标定与验证，不应仅依据一次导航成功推定。